

Day 6 — UDP, TCP vs UDP, Ports, Sockets & Real-World Protocols

Today we move one level up from the TCP lifecycle.

By the end of Day 6, you should be able to explain:

Application

↓

TCP / UDP

↓

Port

↓

IP

↓

Ethernet

↓

NIC

And answer:

Why does an application use TCP instead of UDP?

1. First: TCP vs UDP

You already learned TCP.

TCP provides:

- Connection establishment
- Reliable delivery
- Ordering
- Retransmission
- Flow control
- Congestion control
- Byte-stream abstraction

UDP is much simpler.

TCP

Application

|

▼

TCP

|

Connection
Handshake

UDP

Application

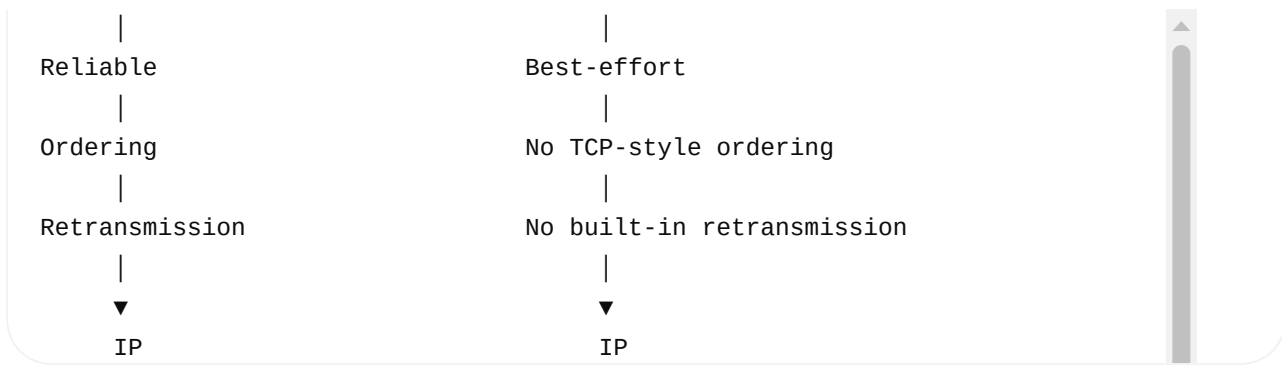
|

▼

UDP

|

No connection
handshake



2. What is UDP?

UDP = User Datagram Protocol

UDP is a **connectionless transport-layer protocol**.

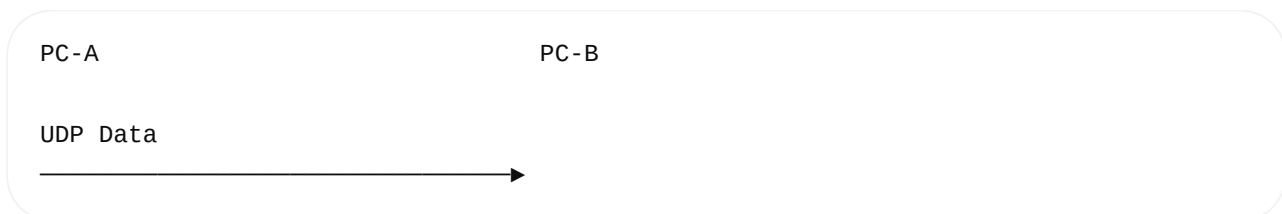
The application gives UDP a datagram and UDP sends it toward the destination.

There is no TCP-style:

SYN
SYN + ACK
ACK

before sending data.

Example:



That's it.

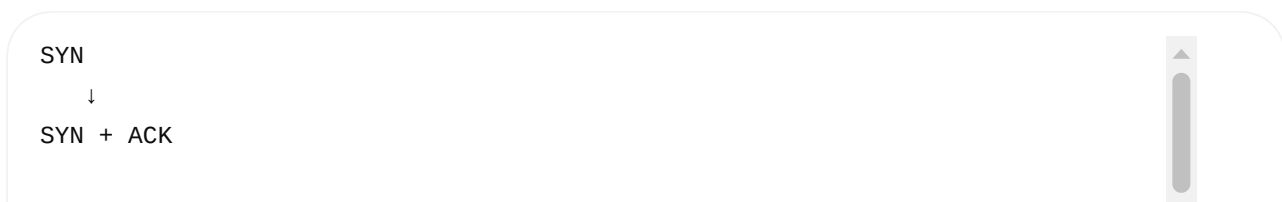
3. TCP vs UDP — Simple Example

Imagine sending a letter.

TCP

You call first:

"Are you there?"





Then:

"Here is my data."

TCP also tracks delivery, ordering, retransmission, etc.

UDP

You simply send:

"Here is my datagram."

DATA →

No connection establishment.

4. Why Would Anyone Use UDP?

Because sometimes you don't want all the mechanisms of TCP.

UDP has:

Less protocol overhead



No connection handshake



Simple datagram delivery



Application controls more of the behavior

Common examples include:

DNS

DHCP

Some real-time media

QUIC/HTTP/3 uses UDP underneath

Important:

UDP itself does **not** guarantee reliable, ordered delivery.

Applications or higher-level protocols can implement additional reliability if needed.

5. TCP vs UDP Table

Feature	TCP	UDP
Full form	Transmission Control Protocol	User Datagram Protocol
Connection	Connection-oriented	Connectionless
Handshake	Yes	No TCP-style handshake
Reliability	Yes	No built-in reliability
Ordering	Yes	No built-in ordering
Retransmission	Yes	No built-in retransmission
Flow control	Yes	No TCP flow control
Congestion control	Yes	No TCP congestion control
Data model	Byte stream	Datagram/message
Header	Larger	Smaller
Typical uses	HTTP/1.1, HTTP/2, SSH, etc.	DNS, DHCP, real-time traffic, QUIC

6. Very Important: TCP is a Byte Stream

Suppose application does:

```
send(sock, "HELLO", 5, 0);  
send(sock, "WORLD", 5, 0);
```

TCP gives the receiving application a **byte stream**.

Conceptually:

```
HELLOWORLD
```

TCP doesn't preserve a message boundary between those two `send()` calls.

This is an important interview point.

7. UDP is Datagram-Based

With UDP:

```
sendto(sock, "HELLO", 5, ...);  
sendto(sock, "WORLD", 5, ...);
```

The datagrams are separate:

Datagram 1

```
| HELLO |
```

Datagram 2

```
| WORLD |
```

The message/datagram boundary is preserved by UDP.

8. Now: What is a Port?

You already know IP addresses.

Example:

```
192.168.1.20
```

This identifies a host/interface address at Layer 3.

But one PC can run many applications.

For example:

PC-B

```
192.168.1.20
```

```
|  
├─ Web server      : 443  
├─ SSH server      : 22  
├─ DNS server      : 53  
└─ Application     : 8080
```

How does the kernel know which application should receive incoming TCP/UDP data?

Port number.

9. IP + Port

Think:

```
IP address
+
Port number
=
Transport endpoint
```

Example:

```
192.168.1.20:8080
```

means:

```
IP    = 192.168.1.20
Port  = 8080
```

The IP gets traffic to the host.

The port helps identify the transport endpoint/application socket on that host.

10. Visualize the Difference

Imagine an apartment building.

```
Building address
  ↓
192.168.1.20

Apartment number
  ↓
8080
```

So:

```
192.168.1.20:8080
```

is like:

```
Building + Apartment
```

Similarly:

```
192.168.1.20:22
```

might be SSH.

192.168.1.20:443

might be HTTPS.

11. Source Port and Destination Port

Suppose your client uses:

PC-A
192.168.1.10
Port = 50000

Server:

PC-B
192.168.1.20
Port = 8080

The TCP connection can be represented as:

192.168.1.10:50000
|
|
▼
192.168.1.20:8080

TCP header contains:

Source Port = 50000
Destination Port = 8080

12. Why Does the Client Usually Have a Random Port?

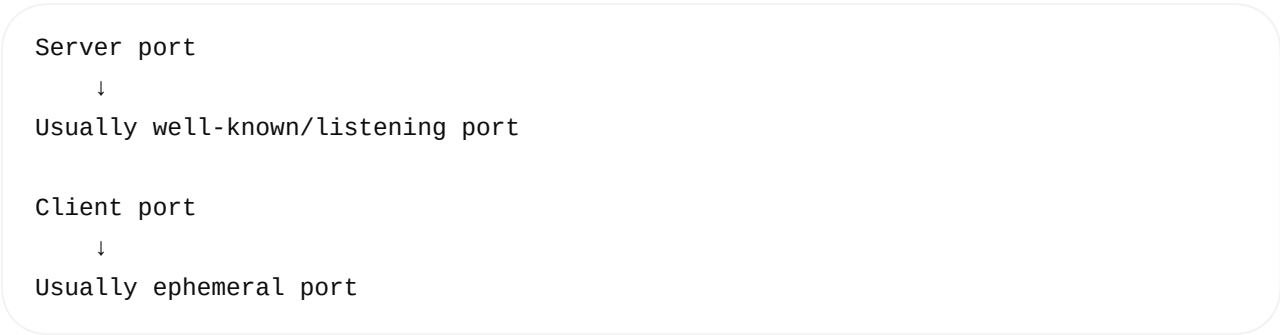
Your application might explicitly bind a client socket, but normally it doesn't need to choose a specific source port.

For example:

Client
192.168.1.10:53124
|
▼
Server
192.168.1.20:8080

The kernel can automatically select an appropriate ephemeral source port.

So:



13. Well-Known Ports

Some common ports:

Port	Protocol/Service	Common use
20/21	FTP	File transfer
22	SSH	Secure remote login
25	SMTP	Mail transfer
53	DNS	Name resolution
67/68	DHCP	Dynamic IP configuration
80	HTTP	Web
443	HTTPS	Secure web
123	NTP	Time synchronization

For interviews, remember especially:

- 22 → SSH

53 → DNS

80 → HTTP

443 → HTTPS

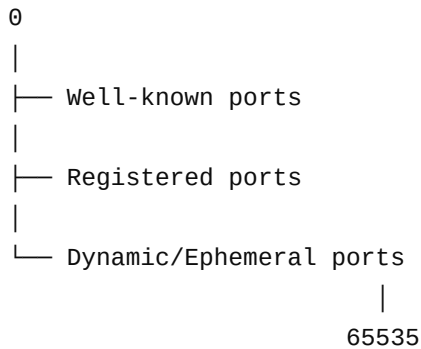
14. Port Range

TCP and UDP ports are 16-bit values.

Therefore:

0 - 65535

Conceptually:



You don't need to memorize every range boundary for a beginner interview unless the interviewer specifically asks.

15. How Does the Kernel Know Which Socket Gets Data?

This is a very important Linux networking concept.

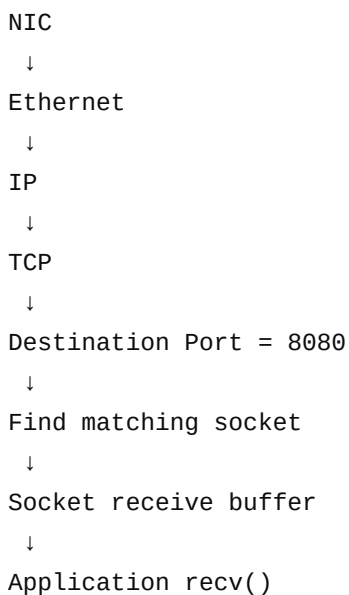
Suppose a TCP segment arrives:

Source:
192.168.1.10:50000

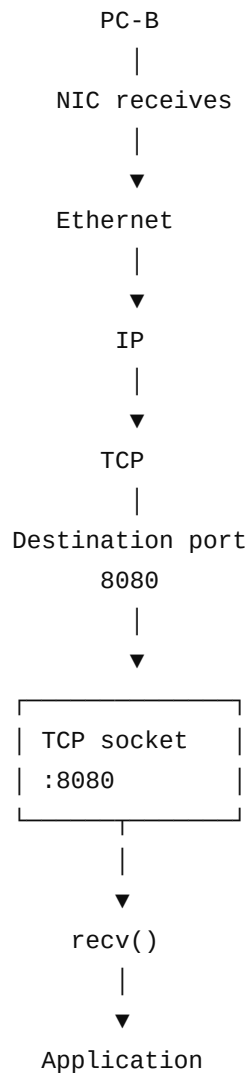
Destination:
192.168.1.20:8080

The kernel's TCP/IP stack processes it.

Conceptually:



Visual:



16. TCP Socket Identification

A TCP connection is commonly identified by the combination:

Source IP
Source Port
Destination IP
Destination Port
Protocol

Example:

TCP

192.168.1.10:50000
↓
192.168.1.20:8080

This is often called the **5-tuple** when protocol is included.

5-Tuple

Source IP
Source Port
Destination IP
Destination Port
Protocol

This is very useful for understanding how multiple connections can exist simultaneously.

17. Multiple Clients to One Server

Suppose:

Server
192.168.1.20:8080

Three clients connect.

Client 1
192.168.1.10:50001

|



192.168.1.20:8080

Client 2
192.168.1.11:50002

|



192.168.1.20:8080

Client 3
192.168.1.12:50003

|



192.168.1.20:8080

All use the same server port:

8080

But the connections are different because the source endpoint is different.

18. TCP vs UDP Socket APIs

This is important because you were learning `socket()` , `bind()` , `connect()` , etc.

TCP

Usually:

```
socket(AF_INET, SOCK_STREAM, 0);
```

UDP

Usually:

```
socket(AF_INET, SOCK_DGRAM, 0);
```

Difference:

SOCK_STREAM

↓

TCP-style stream socket

SOCK_DGRAM

↓

UDP-style datagram socket

19. TCP Server API Flow

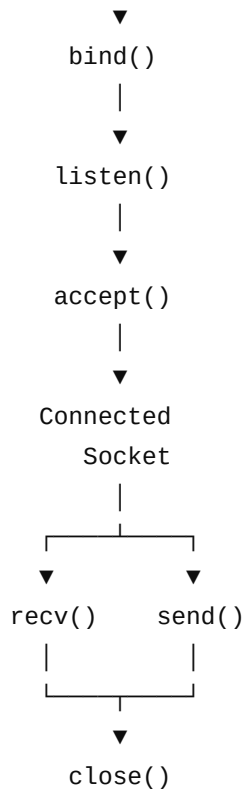
```
socket()  
↓  
bind()  
↓  
listen()  
↓  
accept()  
↓  
recv()/send()  
↓  
close()
```

Visual:

TCP SERVER

socket()
|





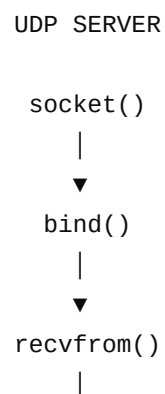
20. UDP Server API Flow

UDP doesn't need `listen()` or `accept()` .

Typical flow:

```
socket()
  ↓
bind()
  ↓
recvfrom()
  ↓
sendto()
  ↓
close()
```

Visual:



```
graph TD; A[process] --> B[sendto()]; B --> C[close()];
```

process

sendto()

close()

21. Why UDP Doesn't Have `accept()`

Because UDP has no TCP-style connection establishment.

TCP:

```
graph TD; C1[Client] -- SYN --> S[Server]; S -- SYN+ACK --> C2[Client]; C2 -- ACK --> Conn[Connection];
```

Client

SYN

Server

SYN+ACK

Client

ACK

Connection

UDP:

```
graph TD; C[Client] -- UDP datagram --> S[Server];
```

Client

UDP datagram

Server

No:

SYN
SYN+ACK
ACK
`accept()`

22. UDP `sendto()` and `recvfrom()`

UDP often needs destination information for each datagram.

Example concept:

```
sendto(sock, buffer, len, 0, &server_addr, ...);
```

Meaning:

Send this datagram to this destination address.

The receiver can use:

```
recvfrom(...)
```

to receive a datagram and learn information about its sender.

23. Now Connect to Real Applications

This is where networking becomes much easier.

You use:

```
graph TD; Browser --> HTTP_HTTPS[HTTP/HTTPS]; HTTP_HTTPS --> TCP_QUIC[TCP or QUIC]; TCP_QUIC --> IP; IP --> Ethernet_Wi-Fi[Ethernet/Wi-Fi]; Ethernet_Wi-Fi --> Network;
```

Browser

↓

HTTP/HTTPS

↓

TCP or QUIC

↓

IP

↓

Ethernet/Wi-Fi

↓

Network

But modern web traffic has an important distinction:

```
HTTP/1.1 → commonly TCP
HTTP/2   → commonly TCP
HTTP/3   → QUIC over UDP
```

So don't memorize:

"HTTP always uses TCP."

That is outdated.

24. What is DNS?

DNS = Domain Name System

Suppose you type:

`www.example.com`

Your computer needs an IP address.

DNS performs name resolution:

`www.example.com`

|



DNS

|



IP address

For example, conceptually:

`www.example.com`

↓

`93.x.x.x`

The exact address can change and shouldn't be memorized.

25. DNS Flow

Very simplified:

Browser/Application

|

"What IP belongs to example.com?"



DNS

|



DNS Server

|

Response



IP address

DNS commonly uses:

UDP port 53

but DNS can also use TCP, and modern DNS-related mechanisms can use other transports.

For beginner interview purposes:

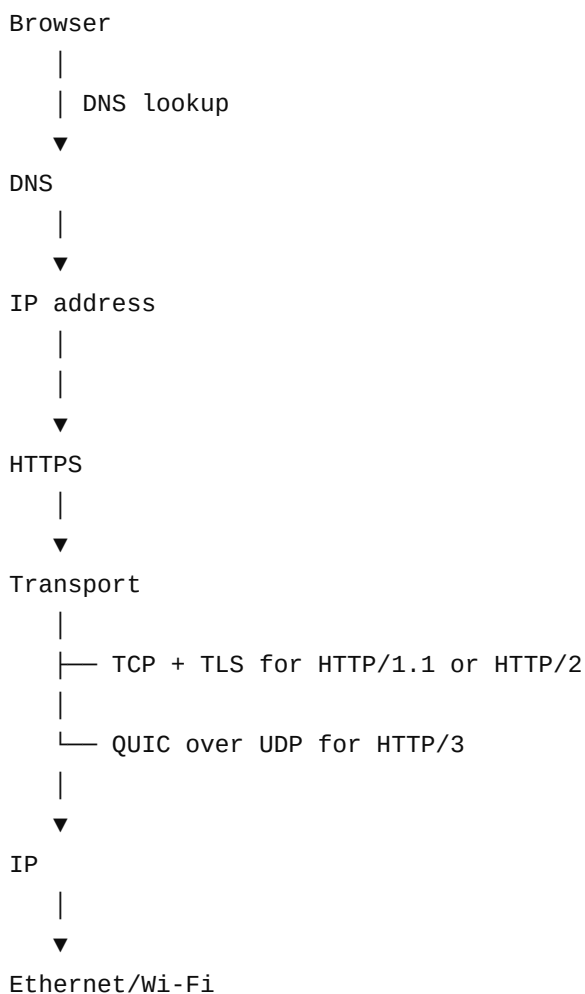
DNS commonly uses UDP port 53 for ordinary queries, but DNS is not limited to UDP.

26. HTTPS Flow

Suppose you enter:

https://example.com

Conceptually:



27. SSH Example

You run:

ssh user@server

Typically:

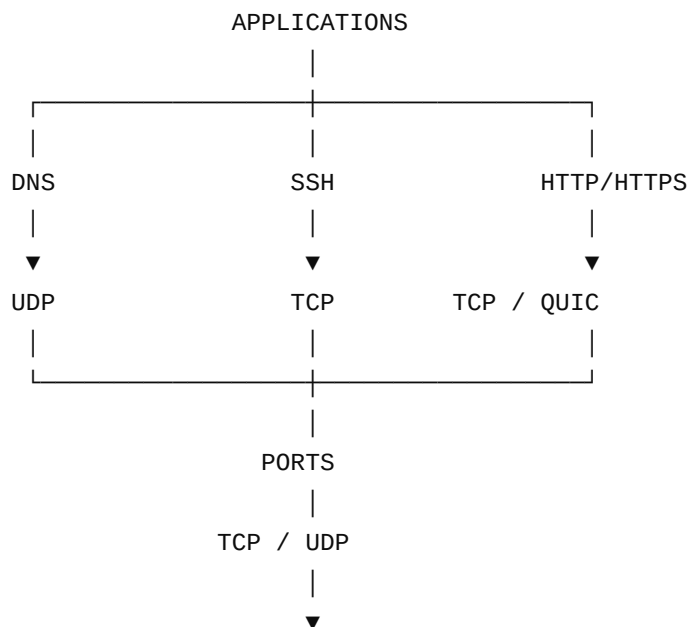
SSH
↓
TCP
↓
Port 22
↓
IP
↓
Ethernet/Wi-Fi

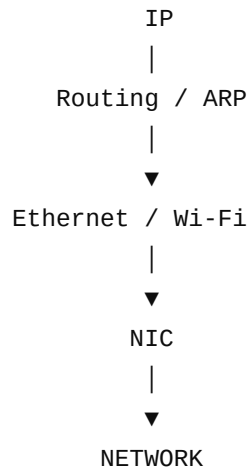
So:

Application
↓
SSH
↓
TCP :22
↓
IP
↓
Ethernet
↓
NIC

28. One Big Picture

This is the diagram I want you to remember from Day 6:





29. The Most Important Mental Model

You have now learned three different things:

Layer 3 — IP

Answers:

"Which host/network should this packet reach?"

IP address
192.168.1.20

Layer 2 — MAC

Answers:

"Which local network interface should receive this frame?"

MAC address
AA:BB:CC:DD:EE:FF

Layer 4 — TCP/UDP Port

Answers:

"Which transport endpoint/application should receive this data?"

TCP port 8080

So:

MAC
↓

Which local interface?

IP

↓

Which host/network?

Port

↓

Which transport endpoint/application?

This is one of the most important networking concepts for your interview.

30. Example: PC-A → PC-B

Suppose:

PC-A

IP = 192.168.1.10

MAC = AA:AA:AA:AA:AA:AA

PC-B

IP = 192.168.1.20

MAC = BB:BB:BB:BB:BB:BB

Server port = 8080

Application:

send()

TCP:

Source Port = 50000

Destination Port = 8080

IP:

Source IP = 192.168.1.10

Destination IP = 192.168.1.20

Ethernet on the local LAN:

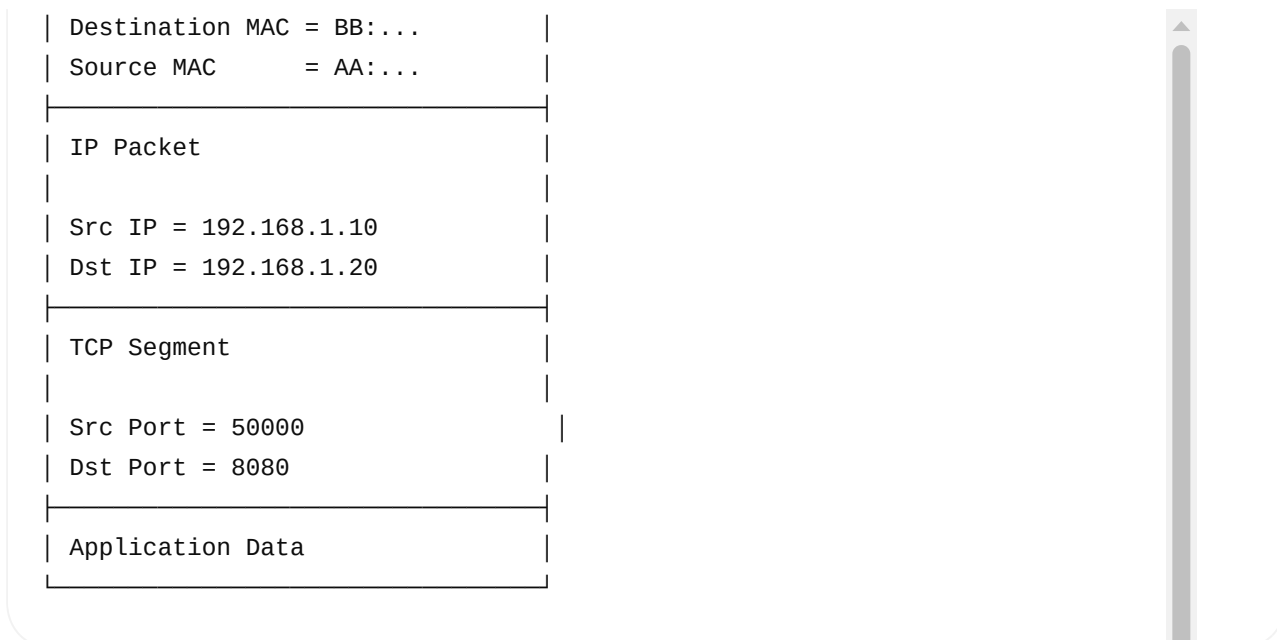
Source MAC = AA:AA:AA:AA:AA:AA

Destination MAC = BB:BB:BB:BB:BB:BB

So conceptually:

Ethernet Frame





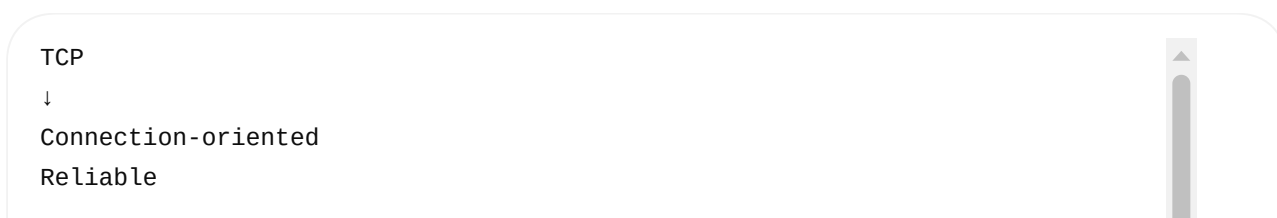
That single diagram connects **Layer 2 + Layer 3 + Layer 4 + application**.

31. Interview Questions for Day 6

Try these yourself:

1. What is UDP?
2. What is the difference between TCP and UDP?
3. Why doesn't UDP need `listen()` and `accept()` ?
4. What is a port number?
5. Why does a client normally use an ephemeral port?
6. What is the difference between IP address and port number?
7. What is the difference between TCP stream and UDP datagram?
8. What is a 5-tuple?
9. What is DNS?
10. What port is commonly associated with DNS?
11. What port is commonly associated with SSH?
12. What port is commonly associated with HTTP?
13. What port is commonly associated with HTTPS?
14. Can HTTP/3 use TCP?
15. Why can multiple clients connect to the same server port?
16. What is the difference between `send()` and `sendto()` ?
17. What is the difference between `recv()` and `recvfrom()` ?

★ Day 6 Must-Memorize



Ordered
Byte stream

UDP
↓
Connectionless
Datagram-based
No built-in reliability
No built-in ordering

IP address
↓
Identifies the network-layer destination

MAC address
↓
Identifies the Layer-2 interface on the local link

Port
↓
Identifies the transport endpoint

And the complete flow:

Application
↓
TCP / UDP
↓
Port
↓
IP
↓
MAC / Ethernet or Wi-Fi
↓
NIC
↓
Network

Day 7 should be the natural next step: ARP + MAC address + Ethernet switching + router forwarding, including the exact difference between "same LAN" and "different network." This will reinforce the Layer 2/Layer 3 questions you've been working on.